

Arduino 演奏システム設計書

25G1051 近藤巧望

2026 年 5 月 12 日

1 システムの基本設計

1.1 機能一覧

トロンボーンユニットの主要機能とその詳細を表 1 にまとめる。実装にあたっては、これらの機能が相互に連携し、リアルタイムな演奏制御を実現することが求められる。

Processing において、トロンボーンの音色は振幅で制御するため F2.1 の振幅ベース音色制御が重要な役割を果たす。音響物理学・音色合成に関する理論では、加算合成法に基づき、基本周波数とその倍音成分の振幅比率を調整することで特定の楽器音色を模倣できることが知られている [1]。

ID	主要機能	機能詳細・役割	対応要素
F1.0	データ同期制御	外部入力値を内部演奏パラメータへ動的にマッピングする。	-
F1.1	パラメータ同期受信	Arduino からの BPM・音名データを変数へ即時反映し同期させる。	currentBpm, currentNote
F1.2	演奏時間動的計算	受信した BPM 値に基づき、音価に対応した実時間を算出する。	beatLength, noteLength
F2.0	音響信号生成系	パラメータ制御による波形合成および信号処理を行う。	-
F2.1	振幅ベース音色制御	各倍音の振幅値（比率）を直接指定し、トロンボーン音色を合成する。	wave1 ~ wave4, amp
F2.2	音階・周波数変換	受信した音名文字列を、DSP 処理用の周波数データへ変換する。	noteToFreq()
F2.3	信号エンベロープ処理	指定された振幅最大値（maxAmp）に基づき、時間的減衰を制御する。	Line (env1 ~ env4)
F3.0	出力・表示	処理結果を視聴覚情報としてユーザへ提示する。	-
F3.1	内部状態表示	BPM, Note 名, 振幅設定値などの稼働パラメータを文字表示する。	draw()

表 1 トロンボーンシステム機能一覧表

1.2 システム構成図

トロンボーンユニットのシステム構成を図 1 に示す。UI/制御系は PC 上のブラウザで実装され、Arduino サーバーと WiFi (HTTP/OSC) を介して通信する。Arduino サーバーは、受信した BPM 情報をもとに全体の演奏テンポを管理し、トロンボーンユニットを含む各楽器ユニットへ同期信号（パルス信号）を小節ごとに送信する。トロンボーンユニットは、受信した同期信号に基づいて自身の演奏タイミングを確定させ、Processing へ演奏開始のトリガーを送信する。Processing は、受信したトリガーに基づいて音響信号（振幅制御による合成音声）を生成し、PC 内蔵スピーカーから出力する。

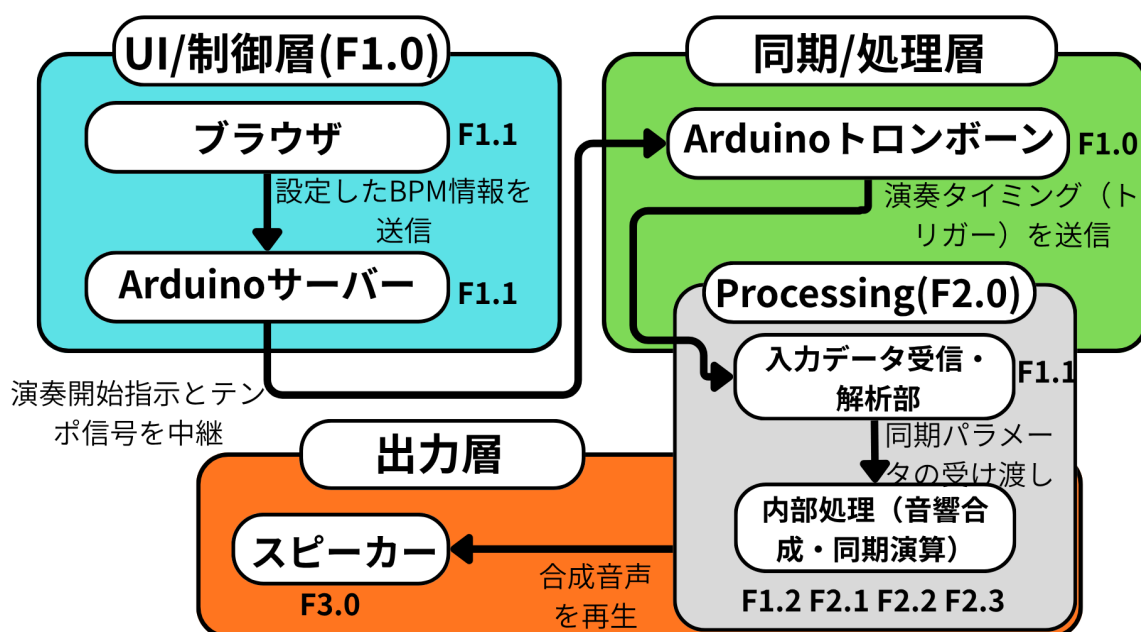


図 1 トロンボーン部分のシステム構成図。表 1 の機能 ID を対応させて記載している。

1.3 操作インターフェース設計

ユーザは、PC 上のブラウザに実装された Web UI を通じてシステムを操作する。本インターフェースは以下の機能を備える。

- **BPM 入力フィールド**：楽曲のテンポを指定し、Arduino サーバーへ WiFi (HTTP/OSC) 経由で送信する。
- **接続ステータス表示**：各 Arduino ユニットとの通信確立状況をリアルタイムで確認できる。
- **演奏制御ボタン**：シーケンスの開始（送信）および強制停止を命令する。

1.4 状態遷移図

本ユニット（Arduino トロンボーン）は、外部機器との同期を自動的に行うため、図 2 に示す 4 つの状態を遷移する。各状態は機能一覧の各項目と密接に関連している。

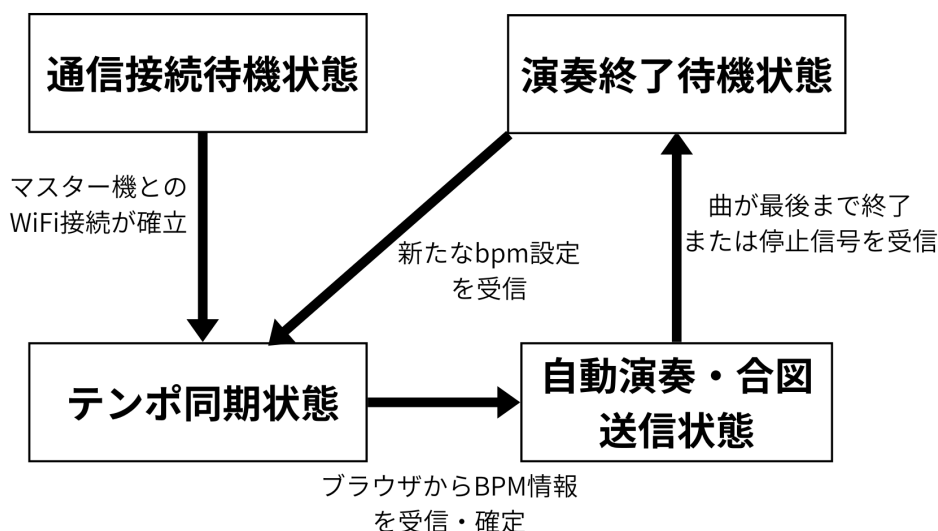


図 2 トロンボーンユニットの状態遷移図

各状態の役割および遷移条件の詳細を以下に示す。

- **1. 通信接続待機状態**：電源投入後の初期状態である。Arduino サーバーとの WiFi 接続を確立し、データの受信準備を整える。接続の確立が次状態へのトリガーとなる。
- **2. テンポ同期状態 (F1.1, F1.2)**：ブラウザ UI から確定された BPM 情報を受信し、内部変数 (currentBpm) を更新する状態である。受信した数値に基づき、音価（音符の長さ）に対応する実時間を計算する。
- **3. 自動演奏・合図送信状態 (F2.0 全般)**：演奏開始信号を受信した瞬間に移行する実行状態である。プログラムされた音階データを処理しつつ、第 1 拍目のタイミングで Processing へ同期トリガーを送信する。この間、振幅制御に基づいた音声合成が並行して実行される。
- **4. 演奏終了待機状態**：楽曲の終了、または手動の停止信号を受信した際の安全状態である。すべての音声出力を停止し、変数をリセットする。新たな BPM 設定を受信することで、再び「テンポ同期状態」へと復帰する。

2 システムの詳細設計

2.1 部品・ソフトウェア一覧

本システムの構築に使用するハードウェアおよびソフトウェア一式を表 2 に示す。まず、前提として OS は macOS を使用する。また、Arduino Uno R4 は WiFi 通信機能を内蔵しているため、外部モジュールは必要ない。スピーカーに関しては PC 内蔵のものを使用するため、特別なオーディオインターフェースも不要である。音声を作成するための Processing では、Minim ライブラリを使用する。Minim は Processing で音響処理をするためのライブラリであり、波形の合成やエンベロープ制御などの機能を提供する [2]。更に、Arduino にプログラム書き込むためには Arduino IDE を使用する。通信プロトコルは、UDP をベースとした OSC (Open Sound Control) を採用する。OSC は音楽やマルチメディアアプリケーション間の通信に特化したプロトコルであり、低遅延かつ柔軟なデータ構造を持つため、本システムのリアルタイムな演奏同期に適している [3]。

分類	名称・型番	用途・備考
ハードウェア	PC	Processing の実行, ブラウザ UI のホスト。
	Arduino Uno R4 WiFi	サーバー機およびトロンボーン機として計 2 台使用 (他楽器を含めると全部で 5 台使用)。WiFi 通信機能必須。
ソフトウェア	Processing	音響合成エンジンおよびメインシーケンサー。
	Minim ライブラリ	Processing 上での波形合成, エンベロープ制御。
	Arduino IDE	マイコン用制御プログラムの開発。
通信プロトコル	UDP / OSC (Open Sound Control)	各ユニット間での低遅延な演奏同期信号の送受信。

表 2 部品・ソフトウェア一覧

2.2 ハードウェア設計

本項では、前節の表 2 に基づき、トロンボーンユニットの具体的な接続関係および動作仕様について述べる。

2.2.1 周辺部品の詳細仕様

本ユニットは、サーバーから受信する小節番号および同期信号に基づき、あらかじめプログラムされたシーケンスを自動実行する構成をとる。そのため、物理入力システム全体のテンポ調整に特化しており、タクトスイッチを用いて BPM の増減を行う。

表 3 使用部品の詳細仕様

名称	型番・メーカー	端子/ピン	特性・役割
BPM 変更ボタン	タクトスイッチ (12mm 角)	D2	演奏テンポ (BPM) を段階的に切り替えるための物理入力。
給電用ケーブル	USB Type-C ケーブル	電源入力	Arduino Uno R4 WiFi への給電およびプログラム書き込み用。

2.2.2 回路構成と接続インターフェース

Arduino Uno R4 WiFi を中心とした接続構成を図??に示す。

具体的な回路接続としては、タクトスイッチの一方を D2 ピン、もう一方を GND に接続する。Arduino 内部のプルアップ抵抗 (INPUT_PULLUP) を有効化し、押下時に LOW となるアクティブロー構成を採用する。

2.2.3 ハードウェア動作の具体化

各部品および通信機能が連携し、演奏同期を実現するプロセスを以下に定義する。

1. **ネットワーク同期による自動演奏開始**：本ユニットは、Arduino サーバーから WiFi (UDP/OSC) 経由で配信される「小節番号パケット」を常時監視する。自身の担当する小節番号を受信した瞬間に演奏フラグを立て、Processing へ音響生成のトリガーを送出する。
2. **物理ボタンによる BPM 同期**：D2 ピンの入力を検知した際、プログラム内の BPM 変数を更新し、その変更を OSC パケットとして他ユニット (サーバー等) へ即座に共有する。
3. **通信モジュールの最適化**：Arduino Uno R4 WiFi の内蔵モジュールを用いることで、外部配線によるノイズや断線リスクを排除し、パケット受信から処理開始までの遅延 (レイテンシ) を最小化している。

2.3 ソフトウェア設計

2.3.1 ファイル構成

- Main.ino：メインループおよびシステム全体の動作状態管理。
- OscParser.cpp / .h：受信した Web OSC メッセージ (BPM, 開始合図) の解析。
- AudioEngine.cpp / .h：BPM に同期したタイマー管理および音響生成ロジック。
- NetworkConfig.h：WiFi 接続情報および外部同期先の IP アドレス定義。

2.3.2 オブジェクト設計 (クラス図)

- **BpmReceiver** クラス：ブラウザから送信された BPM 値を取得し、システム変数へ反映する。
- **PlayManager** クラス：自動演奏のタイミング制御およびメロディデータの読み出しを行う。
- **SyncSender** クラス：演奏開始と同時に Processing 宛での OSC パケットを構築・送出的。

2.3.3 関数設計

- `void receiveBpm()` : UDP ポートを監視し, BPM 値が含まれるパケットを受信した際に内部数値を更新する.
- `void updateTimerInterval(float bpm)` : BPM をタイマー割り込み周期へ変換し, ハードウェアレジスタへ即座に反映する.
- `void broadcastTrigger()` : 演奏の第 1 拍目に合わせ, 無線ネットワーク経由で開始合図を射出する.

2.4 処理フローの設計

本システムは, 以下の 3 段階の自動化フローによって動作する.

1. **初期化** : 起動時に WiFi 接続を確立し, 外部からの OSC パケット受信を待機する.
2. **同期** : ブラウザ UI での操作 (BPM 確定) を受け, 内部の演奏タイマーをマスター機と同期させる.
3. **実行** : 確定したテンポで自動演奏を開始し, 同時に Processing へ向けて「演奏開始合図」を自動送信する.

また, 図 3 に処理フローの概要を示す.

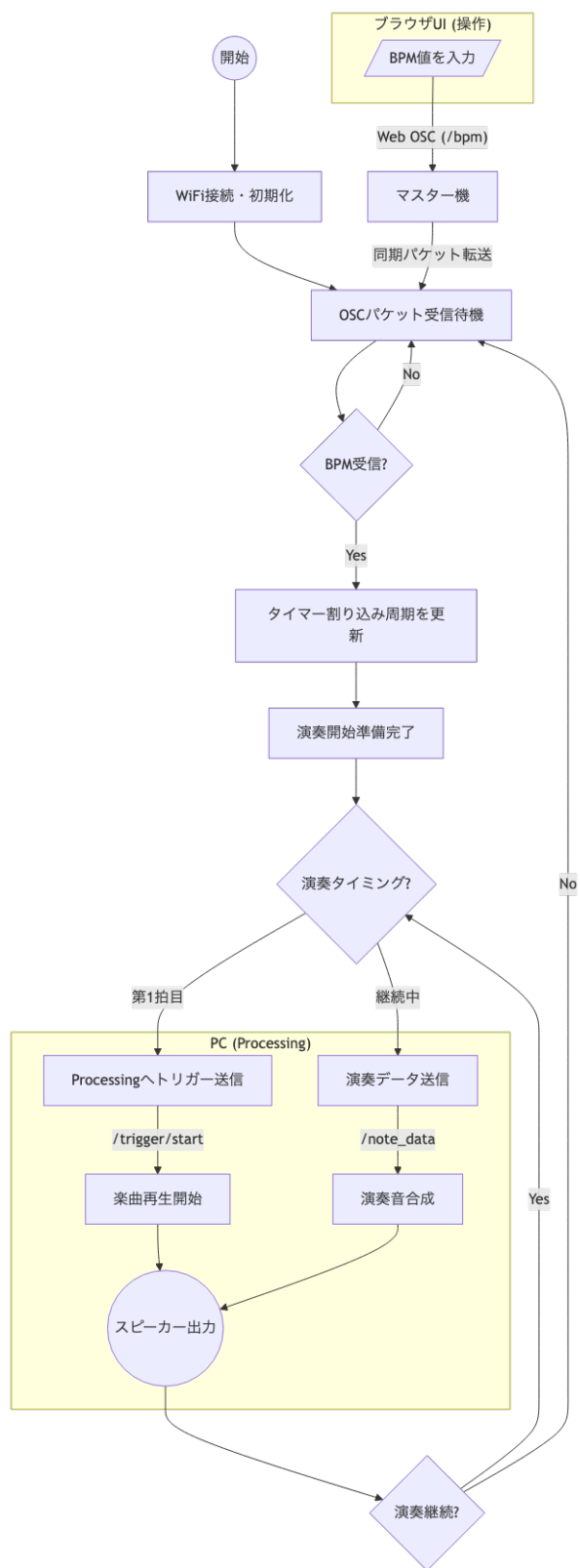


図3 トロンボーン部分の処理フロー図

2.5 テストの設計

- データ整合性テスト：ブラウザ上で入力した BPM 値が，トロンボーン側で正確な演奏周期として反映されているかを確認する。
- 同時再生テスト：トロンボーンの第 1 拍目の発音と，Processing 側の伴奏再生の開始が完全に一致するかを確認する。
- 非接触制御テスト：物理的な接点を一切持たず，ネットワーク経由のデータ通信のみで一連の動作が完結することを確認する。

参考文献

- [1] N. H. Fletcher and T. D. Rossing (2012). The physics of musical instruments. https://books.google.com/books?hl=ja&lr=lang_ja/lang_en&id=gvdSBwAAQBAJ&oi=fnd&pg=PR5&dq=N.+H.+Fletcher+%26+T.+D.+Rossing+%22The+Physics+of+Musical+Instruments%22&ots=noY7enAyw9&sig=St1TtAdA4Qiw9_d_x1au4-K37Mw
- [2] Minim document. <https://code.compartmental.net/minim/>
- [3] Wright, M., & Freed, A. (1997). Open Sound Control: A New Protocol for Communicating with Sound Synthesizers. Proceedings of the International Computer Music. <https://www.adrianfreed.com/sites/default/files/open-soundcontrol-a-new-protocol-for-communicating-with.pdf>